

绵中实验 2020 级信息学竞赛

专题复习——树+数据结构

(考试时间：3.5 小时，8:30—12:00)

温习提示：

- 1、请每位选手认真阅读第一页相关要求。
- 2、所有代码名称必须和程序名称一致。
- 3、程序所有输入输出均采用文件输入输出。
- 4、代码提交：所有程序源代码采用 cena 提交。

题目名称	队栈	文本编辑器	直径	树上统计
程序名称	stqueue.cpp	editor.cpp	diameter.cpp	treecnt.cpp
输入文件	stqueue.in	editor.in	diameter.in	treecnt.in
输出文件	stqueue.out	editor.out	diameter.out	treecnt.out
测试点数	10	10	10	10
测试点分值	10	10	10	10
题目满分	100	100	100	100
时间限制	1s	1s	1s	1s
空间限制	512MB	512MB	512MB	512MB

队栈(staqueue.cpp)

【问题描述】

Yazid 是一名 OI 初学者。他最近在研究基础数据结构：队列和栈。某天，Yazid 脑洞大开，发明了一种叫做“队栈”的数据结构。

众所周知，队列是先进先出的数据结构，而栈是先进后出的数据结构。而 Yazid 发明的队栈则同时支持查询并删除其中最早被插入的元素和最晚被插入的元素。

Yazid 有一个初始为空的队栈和一个黑盒。（这是一个存放数字的盒子），接下来，他要依次执行 Q 个操作。操作分为 4 种：

1. **push**: 将一个非负整数加入队栈。
2. **pop_queue**: 找出队栈中最早被插入的元素，将其取出放入黑盒，并从队栈中删除。
3. **pop_stack**: 找出队栈中最晚被插入的元素，将其取出放入黑盒，并从队栈中删除。
4. **print**: 将黑盒中的所有数取出，并按被放入的先后顺序从左右排列得到一个整数。

• 例如：黑盒中依次被放入了 0,23,330,6 打印的整数即是 233306。

由于 Yazid 懒于在每次 **push** 操作时想插入的数，因此他提前写好了一个长度为 n 的插入序列 A （下标从 1 开始）。在接下来的所有 **push** 操作中，Yazid 会依次地、循环地将这些数加入队栈。具体来说：

- 在首次 **push** 操作时，Yazid 会将 A_1 加入队栈。
 - 在之后的每次 **push** 操作中，假设 Yazid 上次 **push** 时加入的数是 A_i ，则本次他会将 A_{i+1} 加入队栈。
 - 在上一种情况中，特别地，如果 $i = n$ ，则 Yazid 本次会将 A_1 加入队栈。
- 请你依次输出 Yazid 通过 **print** 操作获得的整数。

【输入格式】

第 1 行一个整数 n ，表示插入序列的长度。

第 2 行 n 个用单个空格隔开的非负整数 $A_1, A_2, \dots, A_n$ 描述插入序列。

第 3 行一个整数 Q ，表示操作数目。

第 4 行 Q 个紧挨着的 1~4 之间的数字，依次描述每个操作，每个数字表示操作的编号（各操作对应的编号见【题目描述】）。

- 保证所有操作的合法性。即保证：
 - 执行任意 **pop_queue** 和 **pop_stack** 操作时队栈不为空。
 - 执行任意 **print** 操作时黑盒不为空。

【输出格式】

对于所有 print 操作，输出一行一个整数，表示该 print 操作中 Yazid 获得的数

【输入样例 1】

```
2
1 2
7
1112324
```

【输出样例 1】

```
112
```

【输入样例 2】

```
4
0 23 330 6
12
121313124134
```

【输出样例 2】

```
233306
0
```

【数据范围】

测试点编号	$n=$	$Q=$	其他约定
1	1	0	无
2		10	
3		100	
4~5	10^4	10^5	不存在操作 2
6~7			不存在操作 3
8~9			$A_i > 0$
10		10^7	无

文本编辑器(editor.cpp)

【问题描述】

九发明了一个完美的文本编辑器。这个编辑器拥有两个光标 (cursor)，所以九能够同时 在两地地方插入和删除文本。这个编辑器除了正常的编辑功能以外，还有一些只有九才知道用 处的功能，例如翻转两个光标之间的文本。某一天，九把自己的完美文本编辑器给弄丢了，但 是她还有好多好多文本需要处理。于是她想请聪明又智慧的你帮她实现完美文本编辑器的一些功能。

功能列表如下：

功能名称	命令格式	说明
< (move left)	< w	w 为一个字符，“L”或“R”，表示左光标还是右光标（下同）。 该命令将选定光标向左移动，如果已经是最左端则不移动。 命令执行成功时输出“T”，若光标已经在最左端，则输出“F”。
> (move right)	> w	w 同上。 与< 命令不同的是，该命令将光标向右移动。 命令执行成功时输出“T”，若光标已经在最右端，则输出“F”。
I (insert)	I w c	w 同上。 c 是一个可见字符($33 \leq \text{ascii 码} \leq 126$)，代表在该光标左侧插入该字符。 该命令始终输出“T”。
D (delete)	D w	w 同上。 代表删除该光标右侧的一个字符。 命令执行成功时输出“T”，若光标右侧没有字符输出“F”。
R (reverse)	R	代表翻转左光标和右光标之间的字符。 该命令只有左光标在右光标左侧时才能执行。 (两光标重合时也不能执行) 命令执行成功时输出“T”，否则输“F”。
S (show)	S	代表显示当前处理的文本。 该命令只输出文本，不输出“T”和“F”。

开始时文本编辑器中有一定内容，左光标在第一个字符左，右光标在最后一个字符右。

注意：在插入和删除操作中，没有被操作的光标与文本的相对左右位置保持不变。特别地，若两个光标重叠，操作后也仍然重叠

【输入格式】

第一行是初始时文本编辑器内容。

第二行是一个正整数 N ， N 表示操作次数。

接下来有 N 行，每行有一个命令，命令格式如上方表格

【输出格式】

对于每个命令，按上方表格要求执行并输出。

【输入样例】

```
goodykc
```

11
I R u
I R I
> L
> L
> L
> L
RD
R
< R
D R
S

【输出样例】

T
T
T
T
T
T
T
F
T
T
goodluck

【样例解释】

[goodykc]
[goodykcu]
[goodykcul]
g[oodykcul]
go[odykcul]
goo[dykcul]
good[ykcul]
good[lucky]
good[lucky]（光标右边没有字符， 失败删除）
good[luck]y
good[luck]

Goodluck

【数据范围】

对于 40%的数据: $1 \leq N$, 初始文本长度 ≤ 100 , 数据包含翻转(Reverse)操作;

对于 20%的数据: $1 \leq N$, 初始文本长度 ≤ 100000 , 数据不包含翻转(Reverse)操作;

另有 20%的数据: $1 \leq N$, 初始文本长度 ≤ 100000 , 数据包含翻(Reverse)操作;

对于 100%的数据: $1 \leq N$, 初始文本长度 $\leq 4 \cdot 10^6$, 输出文件大小 $\leq 20\text{MB}$;

树的直径(diameter.cpp)

【问题描述】

小 Q 最近学习了一些图论知识。根据课本，有如下定义。

树：无回路且连通的无向图，每条边都有正整数的权值来表示其长度。如果一棵树有 N 个节点，可以证明其有且仅有 $N-1$ 条边。

路径：一棵树上，任意两个节点之间最多有一条简单路径。我们用 $\text{dis}(a,b)$ 表示点 a 和点 b 的路径上各边长度之和。称 $\text{dis}(a,b)$ 为 a,b 两个节点间的距离。

直径：一棵树上，最长的路径为树的直径。树的直径可能不是唯一的。

现在小 Q 想知道，对于给定的一棵树，其直径的长度是多少，以及有多少条边满足所有的直径都经过该边。

【输入格式】

第一行一个正整数 N ，表示节点数

后面 $N-1$ 行，每行三个正整数 a,b,c ，表示点 a 和点 b 之间有一条长度为 c 的无向边。

【输出格式】

第一行一个正整数表示直径长度

第二行一个整数表示被所有直径经过的边的数量

【输入样例】

```
6
3 1 1000
1 4 10
4 2 100
4 5 50
4 6 100
```

【输出样例】

```
1110
2
```

【样例解释】

直径共两条， $3 \rightarrow 2$ 和 $3 \rightarrow 6$ ，这两条路径都经过边 $(3,1),(1,4)$

【数据范围】

对于 20% 的数据， $N \leq 100$;

对于 40% 的数据， $N \leq 1000$;

对于 100% 的数据， $N \leq 200000, c \leq \text{int}$;

树上统计(treecnt.cpp)

【问题描述】

给定一棵 n 个点的树，现在定义 $Tree[L,R]$ 表示为了使得 $[L,R]$ 中任意两点连通，最少需要选择的边的数量。

现在要求：

$$\sum_{L=1}^n \sum_{R=L}^n Tree[L,R]$$

【输入格式】

第一行一个数 n ，表示树的大小；

后面 $n-1$ 行每行两个数 u,v ，表示一条连接 u 号结点， v 号结点的边。

【输出格式】

输出一个数，表示答案

【输入样例】

```
4
1 4
1 3
2 4
```

【输出样例】

```
16
```

【样例解释】

$Tree[1,1]=0, Tree[1,2]=2, Tree[1,3]=3, Tree[1,4]=3;$

$Tree[2,2]=0, Tree[2,3]=3, Tree[2,4]=3;$

$Tree[3,3]=0, Tree[3,4]=2;$

$Tree[4,4]=0;$

$Tree[L,R]$ 表示点 $L, L+1 \dots R-1, R$ 这些点都连通需要选择的边。

【数据范围】

对于 20% 的数据, $n \leq 10$;

对于 40% 的数据, $n \leq 300$;

对于 60% 的数据, $n \leq 3000$;

对于额外 20% 的数据, 树呈一条链;

对于 100% 的数据, $n \leq 100000$